

VNBOT // TECHNICAL REQUIREMENTS
DOCUMENT // V1.0.0 - DRAFT

VNBOT TRD

Arquitectura técnica, stack, módulos y
criterios operativos para construir VNBOT
como plataforma multiplataforma.

El TRD traduce los objetivos de producto del PRD a una arquitectura ejecutable. Especifica capas del sistema, aplicaciones cliente, servicios backend, persistencia, procesamiento asíncrono, multi-LLM, MCP, audio, memoria de grafo, caching, seguridad técnica, observabilidad, Docker y estrategia de escalabilidad. Referencia para construir VNBOT de forma modular y reproducible.

MODULAR

LOCAL-FIRST

OBSERVABLE

DOCKER-READY

DOCUMENTO

02-TRD-VNBOT.md

VERSIÓN

1.0.0-draft

FECHA

2026-07-16

LICENCIA

MIT (prevista)

Tabla de Contenidos

1. Propósito y alcance	7
2. Principios técnicos	7
2.1. Separación de responsabilidades	7
2.2. El LLM no es una fuente de verdad	8
2.3. Local-first, no local-only obligatorio	8
2.4. Degradación controlada	8
2.5. Operaciones idempotentes	9
2.6. Todo dato sensible tiene alcance	9
2.7. Observabilidad como requisito no funcional	9
3. Arquitectura de alto nivel	9
4. Arquitectura por perfiles de instalación	9
4.1. Perfil Demo — GitHub Pages	9
4.2. Perfil Local — una persona	10
4.3. Perfil Personal Server	10
4.4. Perfil Full	10
5. Stack tecnológico propuesto	11
5.1. Frontend web/PWA	11
5.2. Backend	11
5.3. Base de datos	11
5.4. Cola y jobs	12

5.5. Almacenamiento de objetos	12
6. Clientes y plataformas	13
6.1. Web/PWA	13
6.2. Android APK	13
6.3. Desktop	14
6.4. CLI	14
6.5. Reducción de superficie temprana	14
7. Capas de software	15
8. Módulos técnicos	15
9. Flujo técnico de una instrucción	16
9.1. Estados de operación	16
10. Memoria y grafo	16
10.1. Implementación MVP	16
10.2. Tipos de nodos	16
10.3. Tipos de relaciones	17
10.4. Graphify	17
10.5. Benchmarks de rendimiento del grafo	17
10.5.1. Escenario de referencia	17
10.5.2. Latencia objetivo (P95)	18
10.5.3. Métricas de escala	18
10.5.4. Criterios de aceptación	18
11. Multi-LLM	18

11.1. Interfaz común	18
11.2. Categorías de modelos	18
11.3. Fallback	19
11.4. Seguridad de claves	19
11.5. Presupuesto	19
12. MCP	20
12.1. Función	20
12.2. MCP interno	20
12.3. MCP externo	20
12.4. Seguridad	20
12.5. Transportes	21
12.6. Tool result	21
13. Skills y agentes	21
13.1. Skill	21
13.2. Agente	21
13.3. Niveles de autonomía	22
13.4. Aislamiento	22
14. Procesamiento asíncrono	22
14.1. Operaciones asíncronas	22
14.2. Job mínimo	22
14.3. Reintentos	22
14.4. Scheduler	23
15. Cache	23

15.1. Capas	23
15.2. Cache permitido	23
15.3. Cache prohibido sin protección	23
15.4. Invalidación	24
16. Seguridad técnica	24
16.1. Autenticación	24
16.2. Autorización	24
16.3. Protección de frontend	24
16.4. Protección de backend	24
16.5. Cifrado	25
16.6. Significado de zero-knowledge	25
17. Healthcheck y observabilidad	25
17.1. Endpoints	25
17.2. live	25
17.3. ready	25
17.4. dependencies	26
17.5. Métricas	26
17.6. Tracing	26
18. Docker y orquestación	26
18.1. Docker Compose local	26
18.2. Orden de inicio	27
18.3. Seguridad de contenedores	27
18.4. Kubernetes posterior	27

19. GitHub Pages y Releases	28
19.1. GitHub Pages	28
19.2. Releases	28
20. Compatibilidad y versionado	28
20.1. API	28
20.2. Formato de exportación	29
20.3. Skills	29
20.4. MCP	29
21. Pruebas técnicas	29
21.1. Unitarias	29
21.2. Integración	29
21.3. E2E	30
21.4. Seguridad	30
21.5. Carga	30
22. Estructura de repositorio propuesta	31
23. Requisitos de documentación técnica	31
24. Decisiones técnicas definitivas para la siguiente fase	31
25. Criterios de aprobación del TRD	32
26. Conclusión	33
27. Estrategia de testing	33
27.1. Tipos de tests por capa	34
27.2. Tests obligatorios por PR	34

27.3. Tests obligatorios por release 34

28. Observabilidad 34

28.1. Estándar 34

28.2. Instrumentación mínima por módulo 35

28.3. Dashboards 35

28.4. No recopilar contenido privado 35

1. Propósito y alcance

Este documento define los requisitos técnicos, restricciones, decisiones de arquitectura y criterios operativos para construir VNBOT como una plataforma multiplataforma de memoria personal, recordatorios y mini-agentes. El TRD traduce los objetivos de producto del PRD a una arquitectura ejecutable y verificable.

El TRD especifica los siguientes dominios técnicos:

- Capas del sistema y separación de responsabilidades.
- Aplicaciones cliente (web/PWA, APK, desktop, CLI).
- Servicios backend (API, worker, scheduler, MCP gateway).
- Persistencia (SQLite, PostgreSQL, pgvector, object storage).
- Procesamiento asíncrono (jobs, colas, reintentos, dead-letter).
- Multi-LLM con router, fallback y presupuestos.
- MCP interno y externo con policy engine.
- Pipeline de audio (captura, transcripción, retención).
- Memoria de grafo con procedencia, confianza y benchmarks.
- Caching multinivel con invalidación explícita.
- Seguridad técnica (auth, cifrado, SSRF, CSP).
- Observabilidad con OpenTelemetry desde la primera versión.
- Docker y orquestación para self-hosting.
- GitHub Pages (demo) y GitHub Releases (artefactos).
- Estrategia de escalabilidad progresiva.

El TRD no sustituye el esquema detallado de endpoints, tablas y eventos del Esquema Backend, ni el detalle visual del Diseño UI/UX. Es el documento puente entre el "qué" (PRD) y el "cómo detallado" (Esquema Backend + Plan de Implementación).

2. Principios técnicos

Siete principios técnicos guían todas las decisiones de arquitectura. Cada principio es deliberado y existe porque sin él VNBOT se volvería frágil, opaco o inseguro. Ningún principio es opcional: si una implementación los viola, debe revisarse antes de merger.

2.1. Separación de responsabilidades

La interfaz no debe contener las reglas críticas de negocio. El frontend puede presentar y solicitar operaciones, pero la validación final de fechas, permisos, herramientas, cuotas y persistencia debe estar en una capa de dominio compartida o en el backend. Esta separación garantiza que un cliente comprometido no pueda eludir las reglas.

2.5. Operaciones idempotentes

Cualquier operación que cree, envíe, modifique o elimine datos debe poder reintentarse sin duplicar efectos. Cada operación lleva un `idempotency_key` único que el backend verifica antes de ejecutar. Si la misma key llega dos veces, la segunda llamada retorna el resultado original sin volver a ejecutar el efecto. Esto es crítico para recordatorios recurrentes y para recuperación tras fallos de red.

2.6. Todo dato sensible tiene alcance

Las memorias, archivos, agentes y herramientas pertenecen a un usuario y/o workspace. No existen consultas globales implícitas. Toda query debe especificar el alcance (`user_id`, `workspace_id`) y el backend debe validar que el solicitante tiene permiso sobre ese alcance. Esta regla previene filtraciones accidentales entre usuarios.

2.7. Observabilidad como requisito no funcional

La observabilidad no es un "nice-to-have" post-launch. VNBOT debe poder diagnosticar por qué un recordatorio no se disparó, por qué una búsqueda fue lenta o por qué un MCP falló, sin acceso al contenido del usuario. Los traces, métricas y logs se implementan con OpenTelemetry desde la primera versión, no se añaden después.

3. Arquitectura de alto nivel

La arquitectura de VNBOT sigue un patrón de capas con separación clara entre clientes, API, dominio, persistencia e integraciones. Cada capa tiene responsabilidades exclusivas y se comunica con las adyacentes mediante contratos explícitos (schemas Pydantic, eventos tipados, interfaces de dominio).

Diagrama de la arquitectura de alto nivel de VNBOT.

CLIENTES VNBOT

4. Arquitectura por perfiles de instalación

VNBOT define cuatro perfiles de instalación con conjuntos de componentes distintos. Cada perfil corresponde a un caso de uso real y no requiere componentes innecesarios. La selección del perfil se hace durante el onboarding y determina qué servicios se activan.

4.1. Perfil Demo — GitHub Pages

Muestra la experiencia de VNBOT sin necesidad de backend ni credenciales reales. Está pensado para evaluación inicial del producto y para documentación interactiva.

- React/Vite compilado como sitio estático.
- Datos ficticios en fixtures JSON.
- Chat mock con respuestas pre-grabadas.
- Grafo de ejemplo con 12-20 nodos.
- Mascotas y estados visuales completos.
- Documentación y guías integradas.

GitHub Pages no ejecuta los siguientes componentes:

- API privada.
- Worker.
- Scheduler persistente.
- LLM local.
- Base de datos servidor.
- Secretos.

La demo no debe pedir API keys reales ni almacenar información privada. Todas las respuestas de la demo deben provenir de fixtures o de un modo sandbox explícito.

4.2. Perfil Local — una persona

Adecuado para desktop, terminal y usuarios que no desean un servidor. Toda la infraestructura corre en el dispositivo del usuario.

`vnbot-client<br/>vnbot-local-api<br/>SQLite<br/>worker embebido o proceso local<br/>LLM local opcional`

4.3. Perfil Personal Server

Adecuado para sincronizar varios dispositivos de una persona o una familia pequeña. El servidor corre en una máquina propia (VPS, NAS, mini-PC).

`vnbot-api<br/>vnbot-worker<br/>vnbot-scheduler<br/>PostgreSQL<br/>Redis<br/>MinIO/S3`

4.4. Perfil Full

Adecuado para equipos, comunidades o servidores con varios workspaces. Incluye todos los componentes con escalado horizontal.

`vnbot-api x N<br/>vnbot-worker x N<br/>vnbot-scheduler x 1 o con locks<br/>vnbot-mcp-gateway<br/>postg`

5. Stack tecnológico propuesto

5.1. Frontend web/PWA

Stack recomendado para el frontend web/PWA:

- **React** como librería base.
- **TypeScript** en modo estricto.
- **Vite** como bundler y dev server.
- **TanStack Query** para datos remotos y cache de UI.
- **Zustand** solo para estado de interfaz y sesión local controlada.
- **IndexedDB** mediante **Dexie** para datos locales.
- **CSS/Tailwind** con tokens propios de VNBOT.
- **Canvas/WebGL** opcional para el grafo de memoria.

La aplicación necesita ser estática para GitHub Pages, empaquetable para Capacitor y reutilizable dentro de Tauri. Un frontend desacoplado reduce la dependencia de un servidor Next.js ejecutándose permanentemente y simplifica el despliegue multiplataforma.

5.2. Backend

Stack recomendado para el backend:

- **Python 3.12+** como lenguaje principal.
- **FastAPI** como framework web.
- **Pydantic v2** para validación de schemas.
- **SQLAlchemy 2** como ORM con soporte async.
- **Alembic** para migraciones de schema.
- **Uvicorn/Gunicorn** según el despliegue.

Python facilita el procesamiento de audio, la integración con Whisper, los modelos locales, los workers de IA, las librerías de extracción y el prototipado de memoria y grafos. Si el equipo decide operar completamente en TypeScript, la alternativa es NestJS o Fastify con Drizzle. No se recomienda mantener dos backends principales en lenguajes distintos para las mismas reglas de negocio.

5.3. Base de datos

VNBOT soporta dos motores de base de datos según el perfil de instalación:

Motor	Uso principal	Ventajas
SQLite	Modo local, desktop, instalaciones personales, tests, desarrollo	Sin servidor, file-based, transaccional, ampliamente soportado
PostgreSQL	Servidor, multiusuario, workers concurrentes, pgvector, RLS	Escalado horizontal, full-text search nativa, extensions ecosystem

Regla: ambas implementaciones deben cumplir una interfaz de almacenamiento común. No se debe escribir lógica de negocio específica para PostgreSQL en el núcleo del dominio.

5.4. Cola y jobs

Sistema de cola y jobs para procesamiento asíncrono:

- **Redis** para cola, locks distribuidos y rate limiting.
- **Dramatiq** o **Celery** para workers.
- **Scheduler separado** de la API para evitar duplicación.
- **Dead-letter queue** para jobs fallidos persistentes.
- **Backoff exponencial** en reintentos.

Para una instalación local mínima, puede existir un modo sin Redis basado en una tabla SQLite de jobs, pero este modo no debe presentarse como equivalente a una arquitectura distribuida. Es una alternativa simplificada con límites documentados.

5.5. Almacenamiento de objetos

Almacenamiento de objetos para archivos binarios:

- **MinIO** en self-hosting (compatible con S3).
- **S3 compatible** en instalaciones administradas.
- **Filesystem local** para modo single-user.

Se utiliza para:

- Audios capturados.
- Imágenes y documentos adjuntos.
- Backups cifrados.
- Assets descargables si el proyecto los distribuye.

6. Clientes y plataformas

VNBOT se distribuye en cuatro clientes con contratos API comunes. La reducción de superficie temprana es deliberada: el MVP prioriza Web/PWA y añade los demás clientes solo cuando la sync strategy está probada.

6.1. Web/PWA

El cliente web/PWA debe proporcionar:

- Chat con captura de texto y audio.
- Panel Hoy con recordatorios y accesos rápidos.
- Vista de Memoria y Grafo.
- Gestión de Agentes y Skills.
- Configuración completa.
- Captura de audio mediante permiso explícito.
- Notificaciones web cuando el navegador lo permita.
- Modo offline básico con IndexedDB.

6.2. Android APK

Primera estrategia: **Capacitor**. Permite empaquetar la PWA y añadir capacidades nativas:

- Notificaciones locales.
- Micrófono.
- Filesystem.
- Estado de red.
- Deep links.
- Biometría en una fase posterior.

Limitaciones conocidas:

- Tareas en segundo plano pueden estar limitadas por Android.
- La entrega de notificaciones depende de permisos y optimización de batería.
- La transcripción local puede consumir mucho almacenamiento y RAM.
- Se debe validar la compatibilidad con dispositivos de gama baja.

Requisito: el backend seguirá siendo la fuente de verdad para recordatorios sincronizados. El APK puede mantener recordatorios locales, pero no debe depender exclusivamente de un proceso que Android pueda suspender.

6.3. Desktop

Primera estrategia: **Tauri**. Ofrece ventajas significativas sobre Electron:

- Menor consumo de memoria que Electron.
- Integración con filesystem local.
- Notificaciones nativas del SO.
- Posibilidad de usar SQLite local embebido.
- Empaquetado para Windows, macOS y Linux.

Electron como alternativa si:

- Se requieren módulos Node difíciles de integrar en Tauri.
- Se necesita un ecosistema de plugins desktop más amplio.
- El coste de memoria no es una prioridad.

No se deben mantener Tauri y Electron como productos paralelos en la primera versión. La elección inicial es Tauri y solo se migra a Electron si hay requisitos técnicos que lo justifiquen explícitamente.

6.4. CLI

La CLI debe servir para operaciones de administración y captura rápida:

```
vnbot init<br/>vnbot doctor<br/>vnbot health<br/>vnbot add "revisar presupuesto mañana"<br/>vnbot search
```

La CLI no debe pedir que el usuario pegue secretos en argumentos visibles del historial del shell. Debe aceptar variables de entorno seguras, prompt interactivo o archivos de configuración con permisos restrictivos (0600).

6.5. Reducción de superficie temprana

El TRD original diseña cuatro clientes desde el inicio. Para acelerar el MVP se aplica una estrategia de liberación progresiva:

- **0.1:** Solo Web/PWA. Es el cliente con mayor alcance y menor fricción.
- **0.2:** Se añade soporte Docker para autoalojamiento (es deployment, no un cliente nuevo).
- **0.3:** Se evalúa APK (Capacitor) y Desktop (Tauri) después de tener la sync strategy probada.
- **CLI:** Se implementa como wrapper sobre la API HTTP desde 0.1. Un script vnbot que hace curl a localhost es suficiente para empezar.

No se diseñan adapters de plataforma ni abstracciones prematuras hasta que exista al menos un cliente funcional. Premature abstraction es más costosa que refactor posterior.

7. Capas de software

VNBOT sigue una arquitectura en cuatro capas con responsabilidades exclusivas. Cada capa se comunica solo con las adyacentes mediante contratos explícitos. Esta separación es lo que permite intercambiar implementaciones (PostgreSQL por SQLite, OpenAI por Ollama) sin reescribir el dominio.

Capa	Responsabilidades	No debe hacer
Presentación	Renderizar vistas, recibir interacción, mostrar estado, gestionar cache de UI, mostrar errores comprensibles	Ejecutar reglas críticas de autorización o validación de negocio
Aplicación	Coordinar casos de uso, crear operaciones, enviar jobs, resolver confirmaciones, coordinar transacciones, exponer comandos y queries	Contener reglas de dominio puras (delegar al dominio)
Dominio	Reglas de recordatorios, estados de memoria, reglas de recurrencia, entidades y relaciones, políticas de riesgo, idempotencia, validaciones de negocio	Depender de infraestructura concreta (SQL, Redis, proveedor LLM específico)
Infraestructura	SQL, Redis, LLM providers, MCP transports, filesystem, S3/MinIO, notificaciones, audio	Contener lógica de negocio (solo adaptadores)

8. Módulos técnicos

El backend de VNBOT se organiza en ocho módulos técnicos con responsabilidades delimitadas. Cada módulo tiene su propia interfaz, tests y documentación. Los módulos se comunican mediante eventos y comandos tipados, no mediante llamadas directas acopladas.

Módulo	Responsabilidades clave
Auth y sesión	Registro local opcional, login, sesiones revocables, cookies HttpOnly, Argon2id, MFA posterior, diferenciación entre sesión de cuenta y desbloqueo de bóveda.
Chat Orchestrator	Normalización, clasificación, recuperación de contexto, selección de skill, extracción estructurada, validación, confirmación, ejecución, auditoría.
Memory Engine	Nodos, aristas, procedencia, confianza, sensibilidad, expiración, búsqueda textual, búsqueda vectorial, búsqueda por relaciones, correcciones y contradicciones.
Reminder Engine	Reglas de recurrencia, ocurrencias concretas, zona horaria, ventanas de silencio, prioridad, canales, reintentos, idempotencia, delivery log.
LLM Router	Adaptadores por proveedor, modelos locales, fallback, circuit breaker, presupuesto, conteo de tokens, selección por tarea, política de datos.
MCP Gateway	Registro de servidores, transportes stdio y Streamable HTTP, descubrimiento de tools/resources, scopes, policy engine, timeouts, auditoría, healthcheck.

Módulo	Responsabilidades clave
Audio Pipeline	Captura, carga, normalización, transcripción, revisión, extracción, retención/borrado.
Notification Service	Canales: web, desktop, Android, email (posterior), Telegram (posterior). Cada canal con estado de entrega y reintentos independientes.

9. Flujo técnico de una instrucción

Cada instrucción del usuario atraviesa 16 pasos técnicos desde que llega al cliente hasta que se audita el resultado. Este flujo es el corazón operativo de VNBOT y debe cumplirse sin atajos. Cualquier omisión de un paso es un bug arquitectónico.

1. Cliente envía mensaje
2. API valida sesión y tamaño
3. Se crea operation_id
4. Se guard

9.1. Estados de operación

Cada operación pasa por una máquina de 11 estados. Las transiciones son explícitas y auditables. Una operación no debe pasar directamente de una respuesta textual del LLM a un efecto externo irreversible.

RECEIVED
CLASSIFYING
NEEDS_CLARIFICATION
PROPOSED
WAITING_CONFIRMATION
QUEUED

10. Memoria y grafo

10.1. Implementación MVP

La implementación inicial del Memory Engine usa:

- Tablas **memory_nodes** y **memory_edges**.
- **PostgreSQL** + **pgvector** en servidor.
- **SQLite** + **índice local** en instalación personal.
- Profundidad máxima visible configurable.
- **Top-K** por relevancia.
- Procedencia obligatoria.
- Confianza separada de importancia.

10.2. Tipos de nodos

El Memory Engine reconoce 11 tipos de nodos:

person
place
project
task
reminder
event
preference
note
document

10.3. Tipos de relaciones

Las aristas del grafo pueden ser de 11 tipos:

KNOWS
WORKS_ON
RELATED_TO
DEPENDS_ON
REMINDER_FOR
HAPPENS_AT
PREFERS
SUPER

10.4. Graphify

Graphify será un adaptador opcional para información estructural de repositorios y otros datos compatibles. No reemplaza el Memory Engine personal. VNBOT debe poder:

- Conectarse mediante MCP.
- Mostrar estado y scopes.
- Consultar datos autorizados.
- Crear referencias cruzadas.
- Mantener separadas las memorias personales y las de código.

10.5. Benchmarks de rendimiento del grafo

VNBOT debe definir objetivos de rendimiento para la memoria de grafo antes de la fase de implementación (0.4/0.5). Sin benchmarks, no se puede medir regresiones ni decidir cuándo una optimización es necesaria.

10.5.1. Escenario de referencia

Parámetro	Valor objetivo
Nodos totales	5,000
Edges totales	10,000
Profundidad máxima de consulta	3
Top-K por consulta	20

10.5.2. Latencia objetivo (P95)

Operación	Local (SQLite)	Servidor (PostgreSQL)
Crear nodo	< 50ms	< 30ms
Crear edge	< 50ms	< 30ms
Búsqueda textual (full-text)	< 100ms	< 80ms
Búsqueda semántica (vector)	< 200ms	< 100ms
Subgrafo (profundidad 2, 20 nodos)	< 300ms	< 150ms
Subgrafo (profundidad 3, 50 nodos)	< 500ms	< 250ms
Recorrido con filtro de confianza	< 400ms	< 200ms
Invalidación por borrado	< 200ms	< 100ms

10.5.3. Métricas de escala

- El sistema debe degradar graciosamente (paginación, límite de profundidad) si una consulta excede los umbrales.
- Nunca cargar el grafo completo en memoria del cliente.
- Implementar EXPLAIN ANALYZE o equivalente en tests de integración para detectar regresiones de queries.
- Benchmark automatizado en CI para las operaciones P0.

10.5.4. Criterios de aceptación

- Los benchmarks se ejecutan en CI como parte del pipeline.
- Cualquier commit que degrade P95 en más de 20% en una operación P0 debe ser revisado.
- Los resultados se publican en cada release notes.

11. Multi-LLM

11.1. Interfaz común

Todos los proveedores LLM implementan una interfaz común. Esto permite cambiar de proveedor sin tocar el dominio y añadir nuevos adaptadores sin romper los existentes.

```
class LLMProvider(Protocol):  
    ...  
    async def generate(  
        ...  
    ):
```

11.2. Categorías de modelos

Cada tarea usa el modelo más adecuado según su naturaleza:

12. MCP

12.1. Función

MCP será el sistema de expansión de VNBOT. Permitirá conectar herramientas y recursos de forma estandarizada sin acoplar el núcleo a cada servicio. MCP es el equivalente a un sistema de plugins con permisos granulares y auditoría.

12.2. MCP interno

Herramientas controladas por VNBOT (siempre disponibles):

`memory_search<br/>memory_create<br/>memory_update<br/>memory_forget<br/>memory_link<br/>graph_expand<br/>`

12.3. MCP externo

Conectores posibles mediante servidores MCP externos:

- Graphify (repos de código).
- Calendarios (Google, ICS, CalDAV).
- Email (Gmail, IMAP).
- Filesystem (carpetas locales).
- Web/search.
- Notas (Obsidian, Notion).
- Mensajería oficial (Telegram, WhatsApp Business).

12.4. Seguridad

MCP no concede autorización automáticamente. El gateway valida 10 dimensiones antes de ejecutar cualquier tool call externo:

- Servidor confiable (allowlist).
- Usuario autenticado.
- Workspace válido.
- Agente con permiso.
- Herramienta habilitada.
- Scope aprobado.
- Riesgo calculado.
- Confirmación del usuario si es alto riesgo.
- Presupuesto disponible.
- Timeout configurado.

12.5. Transportes

- **stdio**: servidores locales (ejecutable en misma máquina).
- **Streamable HTTP**: servidores remotos (HTTPS con auth).
- **SSE**: solo cuando sea necesario por compatibilidad con servidores legacy.

12.6. Tool result

Toda herramienta debe devolver resultado estructurado para auditoría:

```
{<br/>&nbsp;&nbsp;&nbsp;"ok": true,<br/>&nbsp;&nbsp;&nbsp;"tool": "calendar.create_event",<br/>&nbsp;&nbsp;&nbsp;"data":
```

13. Skills y agentes

13.1. Skill

Una skill es una capacidad versionada que define:

- Objetivo.
- Entrada esperada.
- Salida producida.
- Herramientas que puede usar.
- Memoria permitida.
- Riesgo calculado.
- Confirmación requerida o no.
- Límites de uso.

13.2. Agente

Un agente combina los siguientes elementos:

- Prompt de sistema.
- Modelo LLM asignado.
- Skills habilitadas.
- Herramientas disponibles.
- Scopes de memoria accesible.
- Nivel de autonomía (0-4).
- Presupuesto.
- Mascota visual asignada.

13.3. Niveles de autonomía

Los agentes tienen 5 niveles de autonomía creciente:

0 – Solo responder
1 – Proponer
2 – Ejecutar acciones internas
3 – Usar integraciones con

13.4. Aislamiento

Cada agente debe tener un contexto mínimo. No debe recibir todas las memorias del usuario por defecto. El sistema debe recuperar solamente el contexto necesario para la skill activa. Este principio de mínimo contexto reduce el riesgo de filtración de información sensible a proveedores LLM externos y mejora el rendimiento.

14. Procesamiento asíncrono

14.1. Operaciones asíncronas

Las siguientes operaciones se procesan de forma asíncrona mediante jobs:

- Audio (transcripción).
- OCR (extracción de texto de imágenes).
- Embeddings (vectorización de memorias).
- Consolidación (merge de memorias duplicadas).
- Briefings (resúmenes diarios/semanales).
- Sincronización MCP.
- Notificaciones (delivery multi-canal).
- Backups (exportación cifrada).
- Importaciones grandes.
- Reindexación.

14.2. Job mínimo

Estructura mínima de un job en cola:

```
{<br/>&nbsp;&nbsp;&nbsp;"id": "job_123",<br/>&nbsp;&nbsp;&nbsp;"type": "send_reminder",<br/>&nbsp;&nbsp;&nbsp;"status":
```

14.3. Reintentos

Política de reintentos según tipo de error:

Tipo de error	Acción
Error de red	Retry con backoff exponencial

Tipo de error	Acción
Error de autenticación	Failed y pedir reconexión al usuario
Entrada inválida	Failed no retryable
Rate limit (429)	Retry con Retry-After header
Proveedor caído	Circuit breaker abierto, fallback a secundario
Acción externa ambigua	No retry automático, marcar NEEDS_CLARIFICATION

14.4. Scheduler

Debe existir un scheduler dedicado o un sistema de locks. No se debe iniciar un cron completo dentro de cada réplica de API porque causaría duplicación de ejecuciones.

Para el MVP se recomienda: Scheduler único + Redis locks + workers. Esto evita duplicación sin introducir complejidad de orquestación prematura.

15. Cache

15.1. Capas

VNBOT implementa cinco capas de cache con responsabilidades distintas:

UI cache TanStack Query
Local cache IndexedDB/SQLite
Server cache Redis
Search

15.2. Cache permitido

- Catálogo de modelos LLM.
- Schemas MCP.
- Configuración no secreta.
- Resultados de búsqueda con TTL.
- Embeddings derivados mediante hash seguro.
- Resúmenes compactos.
- Estados de healthcheck de corta duración.

15.3. Cache prohibido sin protección

- Plaintext sensible.
- API keys.
- Tokens.
- Audio original.

- Respuestas completas con secretos.

15.4. Invalidación

Toda modificación de memoria debe invalidar en cascada:

- Búsqueda relacionada.
- Grafo del workspace.
- Embedding anterior.
- Resúmenes dependientes.
- Cache del cliente.

16. Seguridad técnica

16.1. Autenticación

- Argon2id para hashing de contraseñas.
- Sesiones rotatorias con refresh token.
- Cookies seguras (HttpOnly, Secure, SameSite).
- Revocación server-side de sesiones.
- MFA posterior al MVP.
- WebAuthn posterior al MVP.

16.2. Autorización

El backend debe comprobar `user_id`, `workspace_id`, agente, skill y herramienta en cada operación. No confiar únicamente en IDs enviados por el cliente. Toda autorización se verifica contra el estado del servidor, no contra claims del cliente.

16.3. Protección de frontend

- CSP estricta.
- Sanitización de Markdown/HTML.
- No usar `dangerouslySetInnerHTML` sin sanitizar.
- No guardar secretos en `localStorage`.
- Validar MIME, tamaño y extensión de archivos.
- Protección contra XSS y CSRF.

16.4. Protección de backend

- Rate limiting por IP, usuario y endpoint.

- Validación Pydantic en todos los payloads.
- SSRF protection para webhooks y MCP salientes.
- Timeouts en todas las llamadas externas.
- Límites de payload.
- Protección contra replay.
- Logs sin contenido sensible.
- Dependencias fijadas y escaneadas en CI.

16.5. Cifrado

- AES-256-GCM o XChaCha20-Poly1305 para datos en reposo.
- Argon2id para derivación de secretos derivados de contraseña.
- Salt e IV únicos por operación.
- Envelope encryption para backups.
- Versionado del formato cifrado.
- Rotación documentada.

16.6. Significado de zero-knowledge

VNBOT debe documentar tres escenarios sin ambigüedad:

- El modo local estricto puede mantener plaintext fuera de terceros.
- El servidor privado puede procesar plaintext según su configuración.
- Un LLM externo puede recibir contexto. En este caso no se debe prometer zero-knowledge total.

17. Healthcheck y observabilidad

17.1. Endpoints

Endpoints de healthcheck expuestos por la API:

```
GET /health/live<br/>GET /health/ready<br/>GET /health/dependencies<br/>GET /metrics
```

17.2. live

Comprueba que el proceso responde. No debe marcar como fallido un proveedor opcional. Es el check más ligero, usado para saber si el contenedor está vivo.

17.3. ready

Comprueba que el servicio está listo para recibir tráfico:

- Base de datos conectada.
- Migraciones aplicadas.
- Cola accesible.
- Configuración mínima presente.
- Capacidad de escribir un job de prueba, si corresponde.

17.4. dependencias

Debe devolver estado resumido, latencia y versión de cada dependencia, nunca secretos. Usado para diagnóstico por administradores y por el CLI vnbot doctor.

17.5. Métricas

- Latencia de API (P50/P95/P99).
- Latencia de LLM por proveedor.
- Tokens y coste agregado.
- Jobs pendientes y jobs fallidos.
- Entregas de recordatorios.
- Errores de MCP.
- Cache hits/misses.
- Memoria y CPU del worker.
- Uso de almacenamiento.

17.6. Tracing

Cada operación debe tener identificadores de trazabilidad:

- **trace_id**: sigue la operación a través de todas las capas.
- **operation_id**: identificador único de la operación de negocio.
- **job_id**: si es asíncrona.
- Usuario/workspace anonimizado o interno.

Nunca se debe incluir el contenido completo de una memoria en una traza por defecto. Los traces son metadatos operativos, no registros de contenido.

18. Docker y orquestación

18.1. Docker Compose local

El proyecto debe incluir los siguientes archivos para despliegue local:

19. GitHub Pages y Releases

19.1. GitHub Pages

Build estático para la demo y documentación:

- Vite como bundler.
- Fixtures JSON para datos ficticios.
- Service Worker opcional para offline.
- Base path configurable.
- GitHub Actions para deploy automático.
- Sin secretos.
- Sin datos reales.

19.2. Releases

Artefactos previstos por release:

VNBOT-Setup-x64.exe
VNBOT-linux.AppImage
VNBOT-linux.deb
VNBOT-macos.dmg
VNBOT-android

Cada release debe incluir:

- Versión semántica.
- Cambios (changelog).
- Migraciones requeridas.
- Compatibilidad con versiones anteriores.
- Checksums verificables.
- Firma cuando sea posible.
- Advertencias de seguridad.
- Assets y licencias correspondientes.

20. Compatibilidad y versionado

20.1. API

Usar /api/v1. Los cambios incompatibles requieren /api/v2 o una migración documentada. La política de versionado sigue semver: patch para fixes, minor para features compatibles, major para breaking changes.

20.2. Formato de exportación

El formato de exportación debe tener schema_version, manifest, checksums y metadata de cifrado. El sistema debe conservar importadores para al menos una versión anterior para permitir migraciones graduales.

20.3. Skills

Cada skill tiene versión semántica. Un agente debe declarar la versión usada para facilitar reproducibilidad y rollback.

20.4. MCP

Para cada servidor MCP conectado se debe guardar:

- Versión del protocolo negociada.
- Nombre y versión del servidor.
- Capabilities declaradas.
- Fecha del último handshake.
- Scopes aprobados.

21. Pruebas técnicas

21.1. Unitarias

- Resolución de fechas.
- Recurrencias.
- Idempotencia.
- Políticas de riesgo.
- Validación de schemas.
- Cifrado/descifrado.
- Parser heurístico.
- Permisos.

21.2. Integración

- API + base de datos.
- API + Redis.
- Worker + scheduler.
- Notificaciones.

- LLM mock.
- MCP mock.
- Import/export.

21.3. E2E

- Onboarding.
- Crear recordatorio.
- Reiniciar worker.
- Reintentar delivery.
- Editar memoria.
- Eliminar memoria.
- Modo offline.
- APK.
- Desktop.

21.4. Seguridad

- SAST (Semgrep).
- DAST (OWASP ZAP).
- Escaneo de dependencias (npm audit, pip-audit).
- Secret scanning (Gitleaks).
- Pruebas XSS/CSRF.
- SSRF.
- Autorización entre workspaces.
- Replay de webhooks.
- MCP malicioso simulado.

21.5. Carga

Escenarios de carga para validar escalabilidad:

- 10.000 memorias por workspace.
- 1.000 recordatorios activos.
- 100 ocurrencias simultáneas.
- 100 jobs concurrentes en servidor de prueba.
- 100 nodos visibles sin degradar interacción.

22. Estructura de repositorio propuesta

El repositorio sigue una estructura monorepo con separación clara entre aplicaciones, servicios, paquetes compartidos y assets. Esta organización facilita el desarrollo multiplataforma sin duplicar lógica de negocio.

```
vnbot/<br/>├─ apps/<br/>│   ├─ web/<br/>│   │   ├─ desktop/<br/>│   │   └─ android/<br/>│   └─ cli/<br/>
```

23. Requisitos de documentación técnica

Cada módulo debe tener documentación completa:

- README.
- Dependencias.
- Variables de entorno.
- Comandos de desarrollo.
- Healthcheck.
- Pruebas.
- Limitaciones.
- Política de datos.
- Licencia de dependencias relevantes.

Cada integración debe documentar:

- API oficial utilizada.
- Scopes solicitados.
- Datos que lee.
- Datos que escribe.
- Límites de frecuencia.
- Método de revocación.
- Fallback.
- Riesgos y ToS.

24. Decisiones técnicas definitivas para la siguiente fase

16 decisiones cerradas que guían la implementación:

01. Frontend desacoplado con React/Vite.

02. FastAPI como backend de referencia.
03. SQLAlchemy/Alembic para SQLite y PostgreSQL.
04. IndexedDB para PWA; SQLite para local/desktop.
05. Redis + worker en servidor.
06. Scheduler separado con idempotencia.
07. Tauri para desktop.
08. Capacitor para el primer APK.
09. GitHub Pages únicamente para demo/documentación.
10. MCP mediante gateway con policy engine.
11. Graphify como integración opcional.
12. Heurística como fallback obligatorio.
13. No almacenar API keys en localStorage.
14. No usar node-cron como scheduler distribuido.
15. No usar Map in-memory como rate limit de producción.
16. No afirmar zero-knowledge donde exista procesamiento cloud.

25. Criterios de aprobación del TRD

El TRD se considera aprobado cuando el equipo pueda responder afirmativamente a las 12 preguntas:

01. ¿Se puede ejecutar VNBOT localmente sin servicios cloud?
02. ¿Se puede desplegar en Docker con una ruta clara de backup?
03. ¿El mismo dominio funciona en web, APK y desktop?
04. ¿Los recordatorios sobreviven reinicios y reintentos?
05. ¿La arquitectura separa API, workers y scheduler?
06. ¿Los agentes tienen permisos explícitos?
07. ¿MCP está aislado del dominio crítico?
08. ¿Se puede sustituir el proveedor LLM?
09. ¿El usuario puede exportar y eliminar sus datos?
10. ¿El sistema tiene healthchecks y logs útiles sin filtrar contenido privado?

11. ¿La demo de GitHub Pages funciona sin backend?

12. ¿Los assets y dependencias tienen una estrategia de licencia clara?

26. Conclusión

VNBOT requiere una arquitectura modular porque combina memoria, recordatorios, IA, audio, agentes, MCP y distribución multiplataforma. El riesgo principal no es que la tecnología sea insuficiente; es construir demasiadas capacidades dentro de un único proceso y una única interfaz sin separar responsabilidades.

La arquitectura propuesta prioriza:

- Un núcleo de dominio pequeño.
- Persistencia intercambiable.
- Workers durables.
- Scheduler confiable.
- Proveedores LLM reemplazables.
- MCP controlado por políticas.
- Clientes multiplataforma con contratos comunes.
- Distribución reproducible.
- Observabilidad sin invadir la privacidad.

El modelo interpreta, el dominio valida, el worker ejecuta y la auditoría explica lo ocurrido.

Con esta separación, VNBOT puede crecer desde una instalación local individual hasta una plataforma autoalojable con múltiples agentes e integraciones sin sacrificar control, seguridad ni mantenibilidad.

27. Estrategia de testing

VNBOT requiere una estrategia de testing definida desde el día uno. La cobertura mínima varía por capa según su criticidad y estabilidad.

27.1. Tipos de tests por capa

Capa	Tipo	Cobertura mínima	Herramienta
Dominio (memoria, recordatorios, grafo)	Unit	80%	Vitest / pytest
API endpoints	Integration	70%	Supertest / httpx
LLM Router	Unit + Contract	60%	Vitest + mocks
MCP Gateway	Contract	80%	Pact / custom
Workers / Scheduler	Integration	60%	pytest + fixtures
Frontend (Web/PWA)	Unit + Component	60%	Vitest + Testing Library
E2E	E2E	Flujos críticos	Playwright
Seguridad	SAST/DAST	N/A	Semgrep + OWASP ZAP

27.2. Tests obligatorios por PR

- Todos los tests unitarios pasan.
- Tests de integración de módulos afectados pasan.
- Linter y typecheck sin errores.
- Sin violaciones de seguridad en Semgrep.
- Sin secretos en Gitleaks.

27.3. Tests obligatorios por release

- Suite E2E completa pasa.
- Benchmarks del grafo no degradan más de 20%.
- Accesibilidad (axe-core) sin violaciones AA críticas.
- Dependencias auditadas (npm audit / pip-audit).

28. Observabilidad

28.1. Estándar

VNBOT usa OpenTelemetry como estándar de observabilidad. Todos los servicios deben emitir tres tipos de señales:

- **Traces:** para seguir una operación a través de las capas (cliente → API → dominio → storage → LLM/MCP).
- **Metrics:** contadores, histograms y gauges por módulo.

- **Logs:** estructurados en JSON con correlation ID.

28.2. Instrumentación mínima por módulo

Módulo	Spans obligatorios	Métricas obligatorias
API	Cada endpoint	Latencia P50/P95/P99, error rate, request count
Dominio	Cada operación de negocio	Duración, éxito/fallo
LLM Router	Cada llamada a LLM	Tokens in/out, latencia, coste, provider
MCP Gateway	Cada tool call	Latencia, éxito/fallo, tool name
Workers	Cada job	Duración, reintentos, fallos
Scheduler	Cada tick	Jobs ejecutados, jobs encolados
Storage	Cada query lenta (>100ms)	Query duration, connection pool

28.3. Dashboards

- Por release: dashboard con métricas clave del sistema.
- Por módulo: vista detallada de spans y métricas.
- Alertas: definidas a partir de v0.2 para el modo servidor.

28.4. No recopilar contenido privado

Los traces y logs nunca deben incluir el contenido de memorias, mensajes de chat, ni datos personales. Solo se registran metadatos operativos (IDs, duraciones, estados, tipos). Esta regla es no negociable y se valida automáticamente en CI con scanners de PII en logs.